Actividad: Conversión de clases y sus relaciones en código javascript

Introducción

La **Programación Orientada a Objetos (POO)** es un paradigma que permite organizar el código en **objetos**, los cuales representan entidades del mundo real.

JavaScript soporta POO mediante:

- Objetos literales
- Funciones constructoras
- Clases (ES6)

La POO permite:

- Reutilizar código
- Facilitar mantenimiento
- Modelar problemas reales
- Escalar aplicaciones

Conceptos Generales de POO

Objeto: Un objeto es una colección de **propiedades (datos)** y **métodos (funciones)**.

```
const persona = {
    nombre: "Carlos",
    edad: 25,
    saludar: function() {
        console.log("Hola, soy " + this.nombre);
    }
};

persona.saludar();
```

Clase: Una clase es una **plantilla** para crear objetos.

```
class Persona {
    constructor(nombre, edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    saludar() {
        console.log("Hola, soy " + this.nombre);
    }
}

const p1 = new Persona("Ana", 20);
p1.saludar();
```

Encapsulamiento: Permite **proteger los datos** dentro de una clase.

```
class Cuenta {
    #saldo;

    constructor(saldoInicial) {
        this.#saldo = saldoInicial;
    }

    depositar(valor) {
        this.#saldo += valor;
    }

    verSaldo() {
        return this.#saldo;
    }
}

const cuenta = new Cuenta(1000);
cuenta.depositar(500);
console.log(cuenta.verSaldo());
```

Herencia: Permite que una clase herede propiedades y métodos de otra.

```
class Animal {
    constructor(nombre) {
        this.nombre = nombre;
    }

    hacerSonido() {
        console.log("Hace un sonido");
    }
}

class Perro extends Animal {
    hacerSonido() {
        console.log("Ladra");
    }
}

const perro = new Perro("Firulais");
perro.hacerSonido();
```

Polimorfismo: Permite que diferentes clases implementen un mismo método de forma distinta.

```
class Gato extends Animal {
    hacerSonido() {
        console.log("Maulla");
    }
}

const gato = new Gato("Michi");
gato.hacerSonido();
```

Abstracción: Consiste en mostrar solo lo necesario y ocultar detalles complejos.

Ejemplo: una clase Vehículo donde solo se interactúa con métodos como acelerar() sin conocer su lógica interna.

Ejemplos:

1. **Creación de una clase básica**

```
class Estudiante {
    constructor(nombre, nota) {
        this.nombre = nombre;
        this.nota = nota;
    }

    mostrarInfo() {
        console.log(this.nombre + " tiene nota " + this.nota);
    }
}

const estudiante1 = new Estudiante("Luis", 4.5);
estudiante1.mostrarInfo();
```

2. **Uso de Métodos**

```
class Calculadora {
    sumar(a, b) {
        return a + b;
    }

    restar(a, b) {
        return a - b;
    }
}

const calc = new Calculadora();
console.log(calc.sumar(10, 5));
```

3. Encapsulamiento

```
class Usuario {
    #password;

    constructor(nombre, password) {
        this.nombre = nombre;
        this.#password = password;
    }

    validarPassword(pass) {
        return this.#password === pass;
    }
}

const user = new Usuario("admin", "1234");
console.log(user.validarPassword("1234"));
```

4. Herencia

```
class Empleado {
    constructor(nombre, salario) {
        this.nombre = nombre;
        this.salario = salario;
    }

    mostrar() {
        console.log(this.nombre + " gana " + this.salario);
    }
}

class Desarrollador extends Empleado {
    constructor(nombre, salario, lenguaje) {
        super(nombre, salario);
        this.lenguaje = lenguaje;
    }

    mostrarLenguaje() {
        console.log("Lenguaje: " + this.lenguaje);
    }
}

const dev = new Desarrollador("Carlos", 3000, "JavaScript");
dev.mostrar();
dev.mostrarLenguaje();
```

5. Polimorfismo

```
class Figura {
    calcularArea() {
        return 0;
    }
}

class Cuadrado extends Figura {
    constructor(lado) {
        super();
        this.lado = lado;
    }

    calcularArea() {
        return this.lado * this.lado;
    }
}

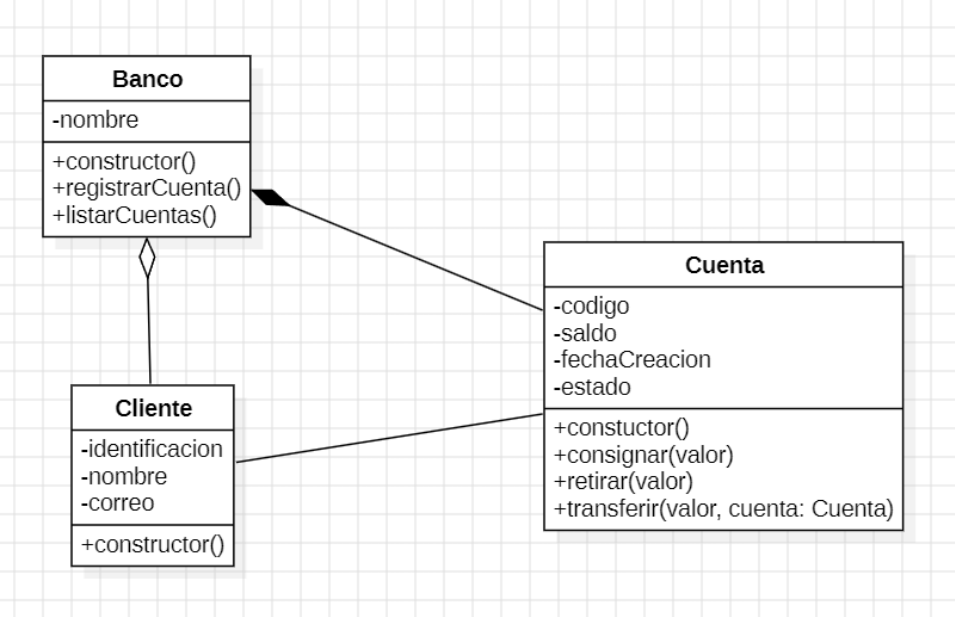
class Circulo extends Figura {
    constructor(radio) {
        super();
        this.radio = radio;
    }

    calcularArea() {
        return Math.PI * this.radio * this.radio;
    }
}

const c1 = new Cuadrado(4);
const c2 = new Circulo(3);

console.log(c1.calcularArea());
console.log(c2.calcularArea());
```

Ejercicio: Convertir en código el siguiente diagrama de Clases



Después de creadas las clases implementarlas teniendo en cuenta los siguientes requerimientos:

- Crear un banco denominado **BANCO ADSO SENA**
- Crear 3 clientes
- Crear una cuenta a cada cliente
- Hacer una consignación a cada una de las cuentas
- Hacer un retiro a cada una de las cuentas
- Hacer un retiro mayor al saldo de una cuenta
- Hacer una transferencia de una cuenta a otra cuenta (solo puede haber transferencia entre cuentas distintas)

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	25-03-2026